

Web Cache Deception Attack

Web Cache Deception Attack - Author not stated, omergil.blogspot.com.

- Published: date not stated
- Original: <<https://omergil.blogspot.com/2017/02/web-cache-deception-attack.html>>
- Preserved from: <https://omergil.blogspot.com/2017/02/web-cache-deception-attack.html> (live) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Did it ever cross your mind that accessing links such as <https://www.paypal.com/myaccount/home/stylesheet.css> or <https://www.paypal.com/myaccount/settings/notifications/logo.png> might expose your sensitive data, and even allow attackers to take control over your account?

Web cache deception is a new web attack vector that puts various technologies and frameworks at risk.

A few words about caching and reactions

1. Websites often tend to use web cache functionality (for example over a CDN, a load balancer, or simply a reverse proxy). The purpose is simple: store files that are often retrieved, to reduce latency from the web server.

Let's see an example of web cache. Website <http://www.example.com> is configured to go through a reverse proxy. A dynamic page that is stored on the server and returns personal content of users, such as <http://www.example.com/home.php>, will have to create it dynamically per user, since the data is different for each user. This kind of data, or at least its personalized parts, isn't cached.

What's more reasonable and common to cache are static, public files: style sheets (css), scripts (js), text files (txt), images (png, bmp, gif), etc. This makes sense because these files usually don't contain any sensitive information. In addition, as can be found in various best practices articles about web cache configuration, it's recommended to cache all static files that are meant to be public, and disregard their HTTP caching headers.

1. The web cache deception attack counts on similar browsers' and web servers' reactions, in the same way as the RPO attack, explained in <http://www.thespanner.co.uk/2014/03/21/rpo/> and <http://blog.innerht.ml/rpo-gadgets/>:

What happens when accessing a URL like <http://www.example.com/home.php/non-existent.css>? A GET request to that URL will be produced by the browser. The interesting thing is the server's reaction – how does it interpret the request URL? Depending on its technology and configuration (the URL structure might need to be built slightly different for different servers), the server returns the content of <http://www.example.com/home.php>. And yes, the URL remains <http://www.example.com/home.php/non-existent.css>. The HTTP headers will be the same as for accessing <http://www.example.com/home.php> directly: same caching headers and same content type (text/html, in this case).

Done with the introduction

What happens if we access <http://www.example.com/home.php/non-existent.css>, while web cache for static files is set on the proxy server, disregarding caching headers for this kind of file? Let's analyze this process:

- Browser requests <http://www.example.com/home.php/non-existent.css>.
- Server returns the content of <http://www.example.com/home.php>, most probably with HTTP caching headers that instruct to not cache this page.
- The response goes through the proxy.
- The proxy identifies that the file has a css extension.
- Under the cache directory, the proxy creates a directory named home.php, and caches the imposter "CSS" file (non-existent.css) inside.

Oh.

!

Taking advantage of it

An attacker who lures a logged-on user to access <http://www.example.com/home.php/logo.png> will cause this page – containing the user's personal content – to be cached and thus publicly-accessible. It could get even worse, if the body of the response contains (for some reason) the session identifier, security answers or CSRF tokens. All the attacker has to do now is to access this page on his own and expose this data.

!

An anecdote

Usually websites don't require authentication to access their public static files. Therefore, the cached files are publicly-accessible – no authentication required.

Conditions

So basically, two conditions are required for this vulnerability to exist:

- Web cache functionality is set for the web application to cache files by their extensions, disregarding any caching header.
- When accessing a page like <http://www.example.com/home.php/non-existent.css>, the web server will return the content of "home.php" for that URL.

Mitigation

- Configure the cache mechanism to cache files only if their HTTP caching headers allow. That will solve the root cause of this issue.
- If the cache component provides the option, configure it to cache files by their content type.

-

Configure the web server so that for pages such as <http://www.example.com/home.php/non-existent.css>, the web server doesn't return the content of "home.php" with this URL. Instead, for example, the server should respond with a 404 or 302 response.

Web Cache Deception in PayPal – PII Exposure

PayPal was vulnerable to web cache deception. The vulnerability is now fixed and was publicly disclosed.

Information that could be leaked by exploiting this vulnerability:

- Users' first & last names
- Account balance
- Last four credit card digits
- Transactions data
- Full passport number
- Email address
- Home address
- Phone number
- Any additional information included in vulnerable pages

Examples for some of the vulnerable pages:

- <https://www.paypal.com/myaccount/home/attack.css>
- <https://www.paypal.com/myaccount/settings/notifications/attack.css>
- https://history.paypal.com/cgi-bin/webscr/attack.css?cmd=_history-details

**** Various static file extensions could be used to cache pages on PayPal (more than 40).**

Among them:

aif, aiff, au, avi, bin, bmp, cab, carb, cct, cdf, class, css, doc, dcr, dtd, gcf, gff, gif, grv, hdml, hqx, ico, ini, jpeg, jpg, js, mov, mp3, nc, pct, ppc, pws, swa, swf, txt, vbs, w32, wav, wbmp, wml, wmlc, wmls, wmlsc, xsd, zip

**** Caching expiration**

I've measured the time taken for the cached files to expire. It seems that after being accessed once (for the first time), a file is cached for ~5 hours. If it's accessed again during that time, the expiration time is extended. It's clear that this time period is more than enough for an attacker to "catch" the cached file on time before it expires, and by constantly monitoring this URL he can expose it as it's created.

Videos

Home page:

<https://www.paypal.com/myaccount/home>

Settings page:

<https://www.paypal.com/myaccount/settings>

History page:

https://history.paypal.com/cgi-bin/webscr?cmd=_history-details

PayPal rewarded me with \$3,000 for reporting this vulnerability.

User Hijacking via Web Cache Deception

I found this vulnerability in additional applications, which unfortunately cannot be disclosed to the public for different reasons (bummer, had some nice videos for that). In these applications, it was possible to **take complete control** over application users. This was possible because the session ID or security answers to recover a user's password were included in the HTML code of vulnerable pages. Big thanks to [Sagi Cohen](#) for the assistance.

IIS Demo

In the video below, a website is hosted on two web servers behind an IIS load balancer with Application Request Routing (ARR) installed. A successful login redirects the users to the 'welcome.php' page, which contains their personal content. The load balancer is configured to cache all CSS files, and to disregard their caching headers.

An authenticated user accesses <http://www.sampleapp.com/welcome.php/stylesheet.css>. The IIS load balancer refers to the 'welcome.php' page as a directory, creates it in the cache directory, and caches 'stylesheet.css', which contains the user's private content.

[Follow @omer_gil](#)